Network Intrusion Detection Using Snort

Category: Network Security | Intrusion Detection | Security Monitoring
Tools: Snort, Kali Linux
Protocols: TCP, HTTP, HTTPS
Devarth Patel

1. Overview

This lab focused on configuring and examining a network intrusion detection system (NIDS) using Snort in a Kali Linux environment.
Snort is an open-source network intrusion detection and prevention system capable of analyzing network traffic and generating alerts when packets match predefined detection rules. For this lab, I configured some custom Snort rules to find TCP traffic associated with HTTP and HTTPS service ports. Then I examined these alerts generated by Snort to understand how signature-based network detection works. The lab introduced practical concepts related to network security monitoring, intrusion detection signatures, and security alert analysis.

2. Objectives

The primary objectives of this lab were to:

- Gain hands-on experience configuring Snort intrusion detection rules.
- Understand the structure and syntax of Snort signatures.
- Create custom alert rules for TCP traffic associated with HTTP and HTTPS service ports.
- Understand how Snort evaluates packets against defined detection conditions.
- Examine security alerts generated when network traffic matches a configured rule.
- Explore the role of intrusion detection systems in network monitoring and security operations.

3. Tools and Environment

Component	**Description**
Operating System	Kali Linux
Intrusion Detection System	Snort
Rule Configuration File	/etc/snort/rules/local.rules
Network Protocol	TCP
HTTP Service Port	80
HTTPS Service Port	443
Detection Method	Signature-based traffic matching

4. Methodology

4.1 Configuring Snort Detection Rules

I began by editing Snort's local rule configuration file in Kali Linux. The local rules file allows us to create custom signatures that define the types of network traffic Snort should monitor. I created two alert rules designed to match TCP traffic originating from source ports commonly associated with HTTP and HTTPS.

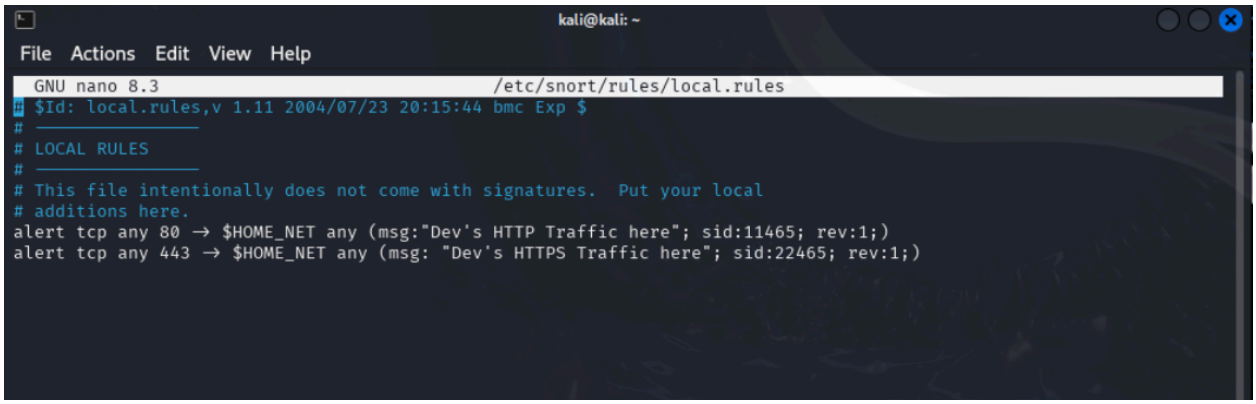


Figure 1: Snort local.rules configuration showing the custom HTTP and HTTPS traffic alert signatures.

4.2 Creating the HTTP Traffic Detection Rule

The first rule was configured to detect TCP packets originating from source port 80 and directed toward the configured home network.

```
alert tcp any 80 -> $HOME_NET any (msg:"Dev's HTTP Traffic here"; sid:11165; rev:1;)
```

This rule has multiple important parts:

Rule Component	**Description**
alert	Instructs Snort to generate an alert when the rule matches
tcp	Specifies the TCP protocol
any 80	Matches any source IP address using source port 80
->	Specifies the direction of matching traffic
$HOME_NET any	Matches any destination port within the configured home network
msg	Defines the message displayed when an alert occurs

sid:11165	Assigns a unique signature identifier
rev:1	Specifies the rule revision

This rule allowed Snort to identify matching TCP packets based on their source port and destination network. Even though port 80 is commonly used by HTTP services, the rule doesn't check HTTP application-layer content. It matches traffic based on the TCP header information.

4.3 Creating the HTTPS Traffic Detection Rule

The second rule was configured to match TCP packets coming in from source port 443.

```
alert tcp any 443 -> $HOME_NET any (msg:"Dev's HTTPS Traffic here"; sid:22465; rev:1;)
```

This rule follows the same structure as the previous rule, but it uses source port 443. Port 443 is normally associated with HTTPS, which uses TLS to protect web communications. This rule was supposed to identify TCP traffic associated with HTTPS services when packets met the specific source-port and destination-network conditions.Since the signature only examines TCP header fields, it doesn't decrypt HTTPS communications or inspect the encrypted app data.

4.4 Examining Snort Alerts

After configuring the rules, I examined Snort's terminal output to see whether the signatures matched the observed network traffic. The terminal displayed multiple alerts associated with the custom HTTPS rule.

The alert output included the configured signature identifier:

```
22465
```

The alerts also displayed the message:

```
Dev's HTTPS Traffic here
```

This shows that Snort recognized packets matching the conditions defined by the HTTPS rule and generated the alert messages accordingly.

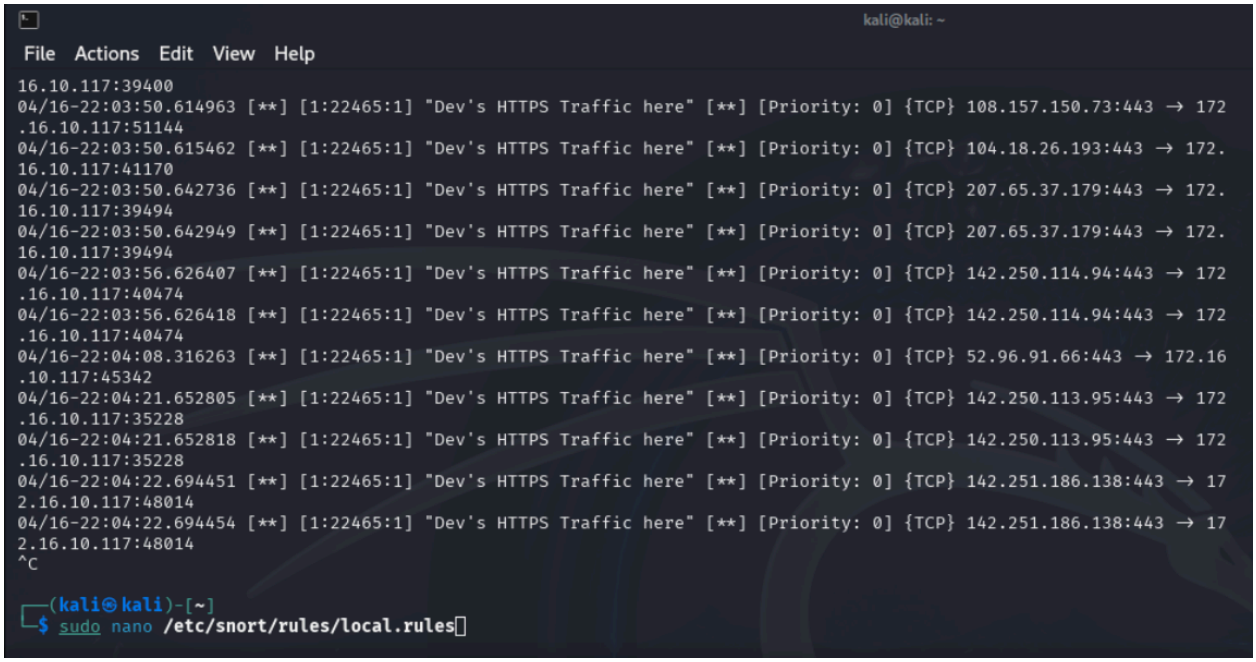


Figure 2: Snort terminal output displaying multiple alerts associated with the custom HTTPS traffic detection rule.

4.5 Understanding the Detection Process

The detection process shown in this exercise can be summarized as follows:

1. Snort checks network packets available through its monitored traffic source.
2. The intrusion detection engine evaluates packets against configured rules.
3. The custom rules check the protocol, source port, destination network, and traffic direction.
4. When a packet satisfies a rule's conditions, Snort generates an alert.
5. The alert contains identifying information that can be reviewed by a security analyst.

This process demonstrates the fundamental operation of signature-based intrusion detection.

5. Results and Findings

The following table summarizes the configured rules and recorded results.

Detection Rule	**Signature ID**	**Matching Condition**	**Recorded Result**
HTTP Traffic Alert	11165	TCP source port 80 toward $HOME_NET	Rule configured
HTTPS Traffic Alert	22465	TCP source port 443 toward $HOME_NET	Multiple alerts observed

The screenshot confirmed that Snort generated alerts using the configured HTTPS signature. The available evidence demonstrates rule configuration and successful matching of at least the HTTPS rule.

6. Cybersecurity Relevance

Intrusion detection systems are important components of network defense because they provide visibility into network activity and can generate alerts based on configured detection conditions. This lab introduced several practical cybersecurity concepts:

- **Network Security Monitoring:** Snort can examine traffic and identify packets that match defined patterns.
- **Signature-Based Detection:** Custom rules allow analysts to specify the network conditions that should trigger alerts.
- **Security Alert Analysis:** Understanding signature IDs, alert messages, and packet information helps analysts interpret detection results.
- **Traffic Classification:** TCP ports, IP addresses, and traffic direction can provide useful context during network investigations.
- **SOC Operations:** Intrusion detection alerts can support security monitoring workflows, although analysts must investigate and validate alerts before determining whether an incident occurred.
- **Detection Engineering:** Writing and testing custom signatures provides foundational experience in designing and evaluating network detection logic.